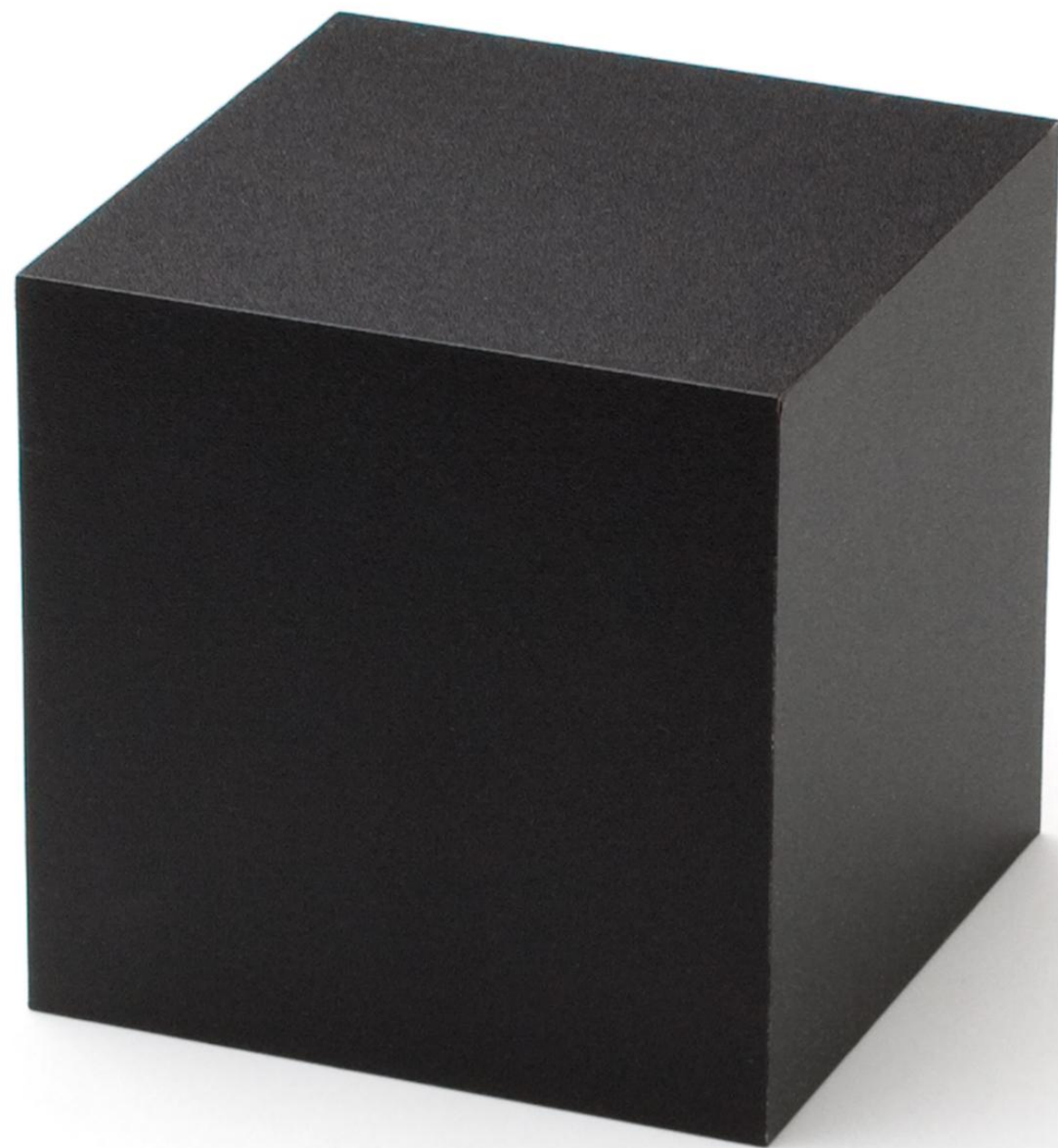




La grande école du numérique pour tous

KickOff Lancement Boîte Noire





Boîte Noire

(1 semaine)



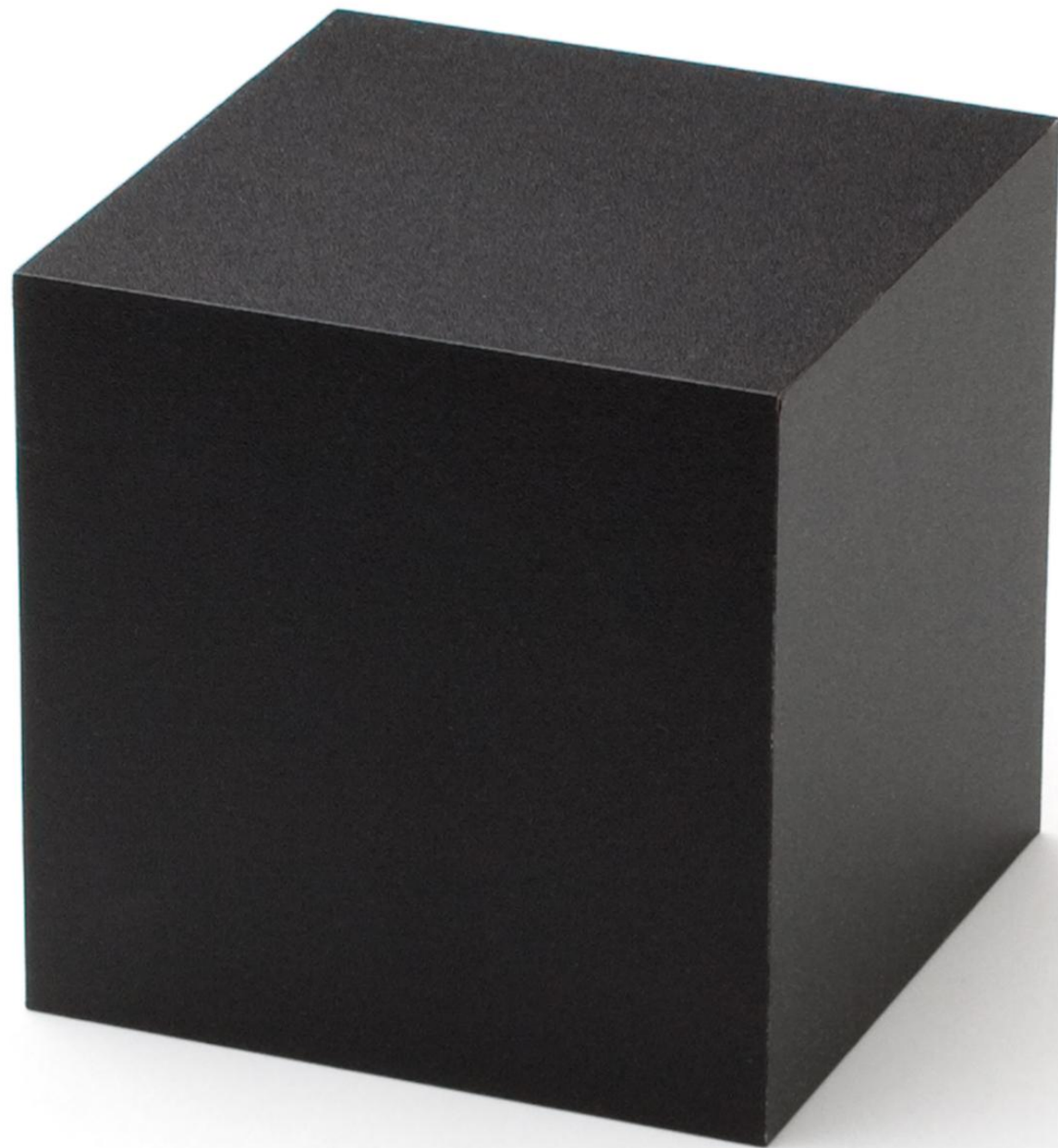
Conception et développement d'un service d'analyse d'événements haute performance pour la messagerie Pigeon.

Stack technique :

- Java
- MongoDB
- Git

Périmètre du projet :

- Pas d'interface graphique (aucune UI)
- Pas de gestion d'utilisateurs ou d'authentification
- Stack ciblée : Spring Boot + Spring Data MongoDB
- Objectif : Répondre instantanément à des questions métier



La problématique

Pigeon compte des dizaines de milliers d'utilisateurs. L'application génère en permanence des logs non exploités :

- Connexions & déconnexions utilisateurs
- Paiements & renouvellements d'abonnements
- Erreurs applicatives & crashes inattendus
- Appels à l'API publique et temps de réponse





Etape 1

Note de décision

(1 Page)

Avant d'écrire la moindre ligne de code, vous devez formaliser par écrit le choix technologique :

- **Limites du relationnel (SQL)**
- **Avantages du documentaire (NoSQL)**
- **Règle d'or Git** : La note doit être commitée avant le premier commit de code Java. L'ordre de l'historique sera rigoureusement contrôlé !



Etape 2 – Schéma Documentaire MongoDB

5+ Types d'Événements

- Concevoir des structures polymorphiques distinctes :
 - connexions
 - paiements
 - requêtes API
 - erreurs
 - notifications
- Nommage 100% en anglais

Embedding vs Referencing

- Justifier formellement les choix selon :
 - la taille
 - la croissance dans le temps
 - la fréquence des requêtes conjointes
- Arbitrage taille / performance

Champs Communs

- Standardiser les champs pivots indispensables pour l'analytique :
 - eventId
 - timestamp
 - userId
 - eventType
- Filtrage transverse fluide



Embedding

Stocke les documents liés **directement à l'intérieur** du document parent (sous forme d'objet ou de tableau d'objets).

- **Principe** : Relation « contient » (composition forte, données lues ensemble).
- **Exemple classique** : L'adresse dans un utilisateur, ou les lignes d'articles dans une commande.

```
// Collection "users"
{
  "_id": "user_123",
  "nom": "Dupont",
  "adresse": {
    "rue": "12 rue de la Paix",
    "ville": "Marseille",
    "code_postal": "13000"
  }
}
```



Referencing

Sépare les entités dans des collections distinctes et tu les relies via un identifiant (l'équivalent d'une clé étrangère en SQL).

- **Principe** : Relations 1-N avec grand volume, relations N-N, ou entités indépendantes réutilisables.
- **Exemple classique** : Des logs rattachés à un compte, ou des produits liés à une catégorie.

```
// Collection "categories"
{
  "_id": "cat_abc",
  "nom": "Informatique"
}

// Collection "produits"
{
  "_id": "prod_789",
  "titre": "Clavier mécanique",
  "categorie_id": "cat_abc"
}
```



Etape 3 – Générateur de données réalistes

- **100K+ Évènements minimum**
- Généré via une classe Java dédiées intégrée au projet

Temporalité 1 An

- Pics en journée
- Creux nocturnes
- Variations semaine/week-end.

Profils Utilisateurs

Loi de Pareto : quelques gros comptes très actifs et une majorité modérée.

Exigences de Réalisme pour l'Analytique

Une distribution uniforme fausse les analyses.

Les données doivent simuler des comportements réels :

Commande Unique

Lancement immédiat par profil Spring ou tâche Maven/Gradle.

100% Java

Scripting externe (Python, Bash) strictement interdit.



Etape 4 – Les 4 pipelines d'agrégation

Tout le calcul doit être délégué à MongoDB.
Le chargement en mémoire Java (boucles) est **formellement interdit**.

Analyses Demandées :

Top 10 utilisateurs :

Classement des utilisateurs les plus actifs sur une période paramétrable.

Répartition Erreurs :

Distribution temporelle des erreurs par type et par jour calendaire.

Performance API :

Temps de réponse par endpoint : calcul de la moyenne et du 95e percentile (p95).

Entonnoir de Conversion :

Séquence d'actions (ex. Inscription → Premier message → Abonnement payant).



Charger 100 000 documents pour les compter dans une boucle est exactement ce qu'il faut éviter.

Etape 5

Optimisation de performance

Analyse Explain Initiale :

Cibler la requête la plus lente parmi les 4 analyses exposées :

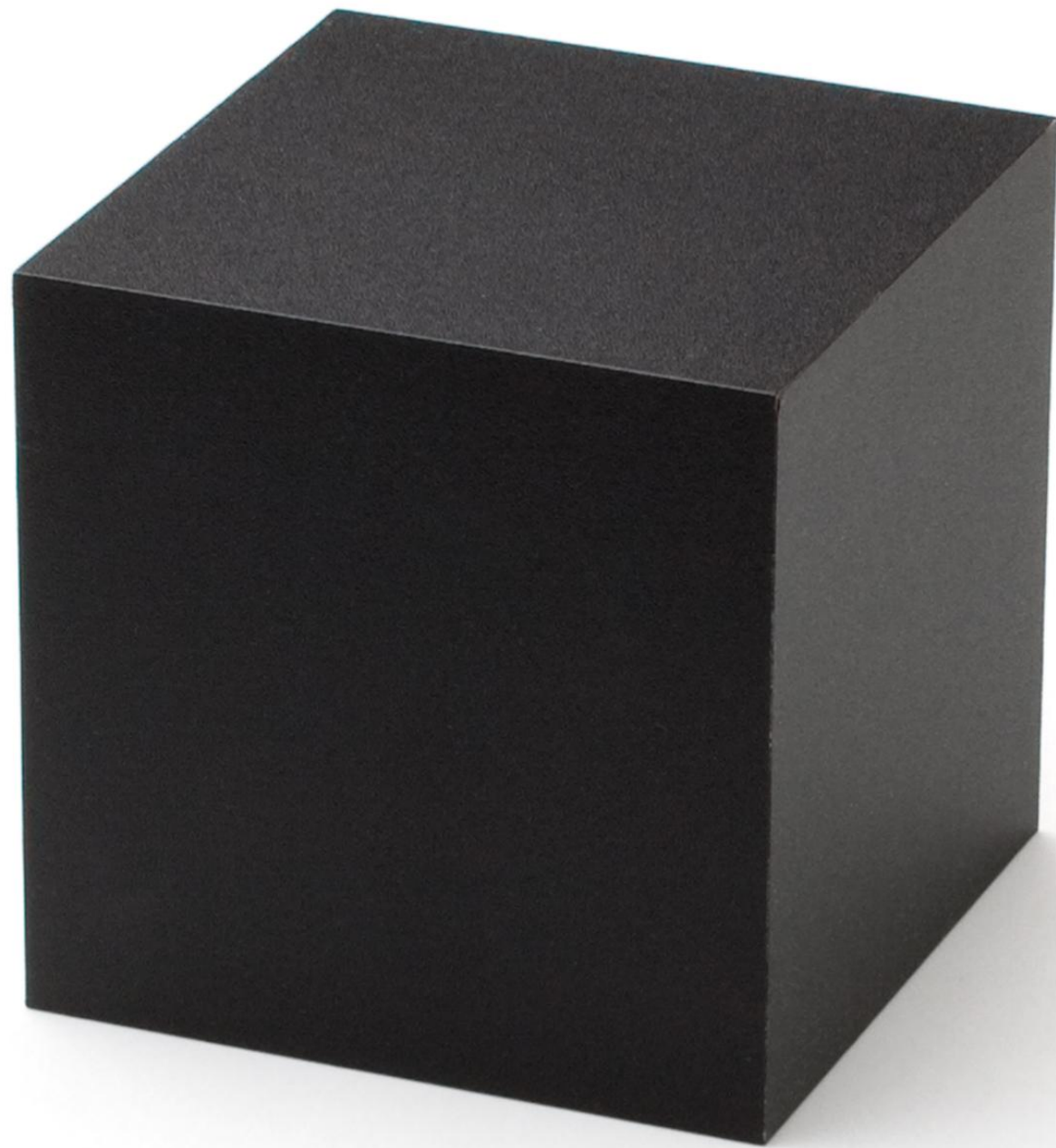
- Exécuter explain("executionStats") avant création d'index
- Relever le temps d'exécution exact (en millisecondes)
- Noter le nombre de documents scannés (totalDocsExamined) vs retournés (COLLSCAN)

Indexation & Gains :

Créer l'index optimal (simple ou composé) et documenter l'effet :

- Justifier l'ordre des champs (règle *Equality, Sort, Range*)
- Refaire la mesure explain avec l'index actif (IXSCAN)
- Rédiger un bilan clair en 3 lignes montrant le gain drastique de vitesse





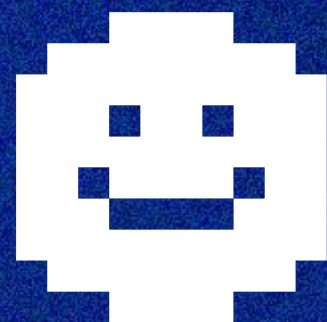
Livrables

Dossier docs/

- **Note de décision** : 1 page (SQL vs NoSQL), commitée avant le code.
- **Schéma documentaire** : Justification de l'embedding/referencing.
- **Dossier de mesure** : Rapport explain avant/après, index et bilan.

Dossier src/& Racine

- **Service Spring Boot** : Endpoints d'analyse documentés sous Swagger.
- **Générateur Java** : Injection des 100k documents en 1 commande.
- **README.md** : Guide complet de lancement et de test.



 La Plateforme